

Lecture 8: Numerical methods for one-dimensional unconstrained optimization

MA3236 Non-Linear Programming

Hu Yukuan

yukuan.hu@nus.edu.sg
S17-08-21

From optimality conditions to computation

Topic 2 characterized stationary points as candidates of local minimizers.

In practice, the stationarity equation

$$\nabla f(\mathbf{x}) = \mathbf{0}$$

rarely has a closed-form solution.

From optimality conditions to computation

Topic 2 characterized stationary points as candidates of local minimizers.

In practice, the stationarity equation

$$\nabla f(\mathbf{x}) = \mathbf{0}$$

rarely has a closed-form solution.

Our computational goal

Generate iterates $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \dots$ that **approach** a (useful) stationary point.

A general iterative framework

Throughout Topic 3, assume f is defined on an open domain $\mathcal{D}$.

① **Initialize:** choose $\mathbf{x}^{(0)} \in \mathcal{D}$.

② **Update:** use available information to construct

$$\mathbf{x}^{(k+1)} = \mathcal{A}^{(k)}(\mathbf{x}^{(0)}, \dots, \mathbf{x}^{(k)}; f, \nabla f, \mathbf{H}_f, \dots) \in \mathcal{D}.$$

③ **Test:** stop if a prescribed termination condition holds; otherwise increase k and go to step 2.

Update through line search

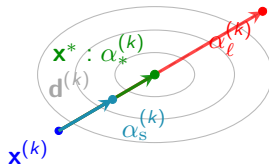
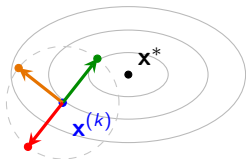
Given past information, choose separately

search direction (**next lecture**): $\mathbf{d}^{(k)} \in \mathbf{R}^n$,

step size (**this lecture**): $\alpha^{(k)} > 0$,

and update

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{d}^{(k)} \in \mathcal{D}.$$



Another way of update: trust region chooses $\mathbf{d}^{(k)}$ and $\alpha^{(k)}$ simultaneously.

One-dimensional line-search subproblem

Once $\mathbf{d}^{(k)}$ is fixed, define

$$\mathcal{I}^{(k)} := \{\alpha \geq 0 : \mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)} \in \mathcal{D}\}, \quad \phi^{(k)}(\alpha) := f(\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)}).$$

- **(Numerically) exact line search (this lecture):**

$$\alpha^{(k)} \in \arg \min_{\alpha \in \mathcal{I}^{(k)}} \phi^{(k)}(\alpha).$$

- **Inexact line search (next lecture):** accept a step satisfying a useful decrease condition, such as Armijo's condition.

Termination tests

- Absolute and scaled residual tests:

$$\begin{aligned}\|\nabla f(\mathbf{x}^{(k)})\| &\leq \epsilon_g \quad \text{or} \quad \epsilon_g \max\{1, \|\nabla f(\mathbf{x}^{(0)})\|\}, \\ \|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| &\leq \epsilon_x \quad \text{or} \quad \epsilon_x \max\{1, \|\mathbf{x}^{(k)}\|\}, \\ |f(\mathbf{x}^{(k+1)}) - f(\mathbf{x}^{(k)})| &\leq \epsilon_f \quad \text{or} \quad \epsilon_f \max\{1, |f(\mathbf{x}^{(k)})|\}.\end{aligned}$$

ϵ_g , ϵ_x , ϵ_f are prescribed positive constants.

- Maximum iteration number: $k \geq K_{\max} \in \mathbb{N}$.
- Maximum computational time: $t \geq T_{\max} > 0$.

Termination tests

- Absolute and scaled residual tests:

$$\begin{aligned}\|\nabla f(\mathbf{x}^{(k)})\| &\leq \epsilon_g \quad \text{or} \quad \epsilon_g \max\{1, \|\nabla f(\mathbf{x}^{(0)})\|\}, \\ \|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| &\leq \epsilon_x \quad \text{or} \quad \epsilon_x \max\{1, \|\mathbf{x}^{(k)}\|\}, \\ |f(\mathbf{x}^{(k+1)}) - f(\mathbf{x}^{(k)})| &\leq \epsilon_f \quad \text{or} \quad \epsilon_f \max\{1, |f(\mathbf{x}^{(k)})|\}.\end{aligned}$$

ϵ_g , ϵ_x , ϵ_f are prescribed positive constants.

- Maximum iteration number: $k \geq K_{\max} \in \mathbb{N}$.
- Maximum computational time: $t \geq T_{\max} > 0$.

For benchmark problems, we may know f^* or $\mathbf{x}^*$.

$$\begin{aligned}|f(\mathbf{x}^{(k)}) - f^*| &\leq \epsilon_f \quad \text{or} \quad \epsilon_f \max\{1, |f^*|\}, \\ \|\mathbf{x}^{(k)} - \mathbf{x}^*\| &\leq \epsilon_x \quad \text{or} \quad \epsilon_x \max\{1, \|\mathbf{x}^*\|\}.\end{aligned}$$

When we talk about an algorithm, we care about its ...

Cost

- Per iteration: (basic) $\times$, $/$, $\mathbf{Ax} = \mathbf{b}$; (oracle) f , ∇f , $\mathbf{H}_f$, ...
- Storage: $\mathbf{x}^{(k)}$, $\nabla f(\mathbf{x}^{(k)})$, $\mathbf{H}_f(\mathbf{x}^{(k)})$, ...
- Total complexity to meet a specified termination test.

Convergence

- Assumption: f , $\mathbf{x}^{(0)}$, ...
- Local/global convergence to a stationary point or an optimizer.
- Local/global convergence rates.

How do we choose an algorithm?

Goal first: e.g., minimize the computational time.

Our budget: computational resource, hardware, ...

Cost

- Per iteration: (basic) $\times$, $/$, $\mathbf{Ax} = \mathbf{b}$; (oracle) f , ∇f , $\mathbf{H}_f$, ...
- Storage: $\mathbf{x}^{(k)}$, $\nabla f(\mathbf{x}^{(k)})$, $\mathbf{H}_f(\mathbf{x}^{(k)})$, ...
- Total complexity to meet a specified termination test.

Convergence

- Assumption: f , $\mathbf{x}^{(0)}$, ...
- Local/global convergence to a stationary point or an optimizer.
- Local/global convergence rates.

$\min / \max \text{ Val}(\{\mathcal{A}^{(k)}\}), \text{ s. t. budget, solutions, implementation, ...}$
 $\{\mathcal{A}^{(k)}\}$

Q-convergence rates

Suppose $\mathbf{x}^{(k)} \rightarrow \mathbf{x}^*$, and let $e^{(k)} = \|\mathbf{x}^{(k)} - \mathbf{x}^*\|$.

- **Q-linear:** for some $c \in (0, 1)$,

$$e^{(k+1)} \leq ce^{(k)} \quad (\text{for all sufficiently large } k).$$

- **Q-superlinear:**

$$\frac{e^{(k+1)}}{e^{(k)}} \longrightarrow 0.$$

- **Q-quadratic:** for some $C > 0$,

$$e^{(k+1)} \leq C(e^{(k)})^2 \quad (\text{for all sufficiently large } k).$$

Lecture 8: Numerical methods for one-dimensional unconstrained optimization

MA3236 Non-Linear Programming

Hu Yukuan

yukuan.hu@nus.edu.sg
S17-08-21

Three one-dimensional methods

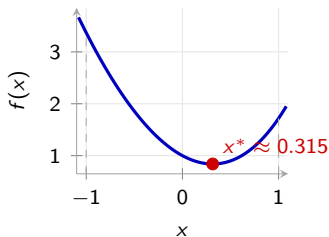
Approximately solve the one-dimensional stationary condition

$$f'(x) = 0.$$

Method	Main data	Immediate target
Bisection on f'	$\text{sign}(f')$	a root of f'
Newton on f'	f' and f''	a root of f'
Golden section	function values	a unimodal minimizer

We compare all three methods on the same objective:

$$f(x) = e^x + x^2 - 2x,$$



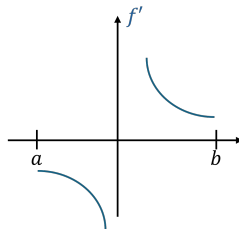
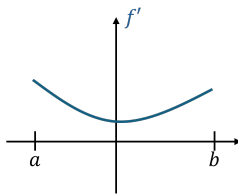
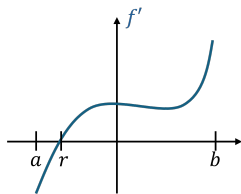
Bisection method: intermediate value theorem

Set $g = f'$. Choose a closed bracket $[a^{(0)}, b^{(0)}] \subset \mathcal{D}$ such that

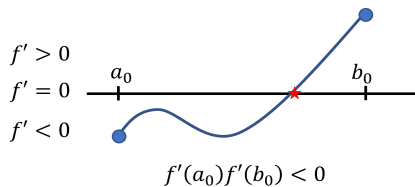
$$g \text{ is continuous,} \quad g(a^{(0)})g(b^{(0)}) < 0.$$

Theorem 8.1 (Derivative bracket)

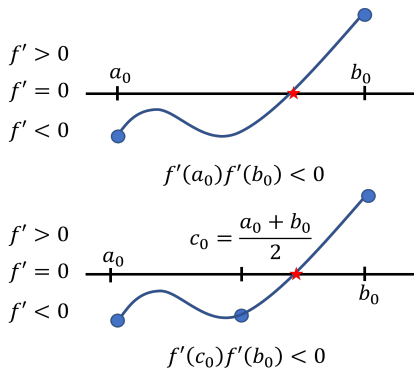
Some $x^ \in (a^{(0)}, b^{(0)})$ satisfies $f'(x^*) = 0$.*



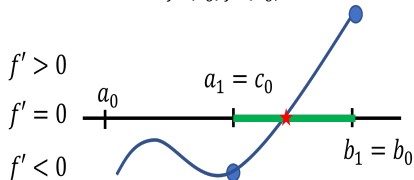
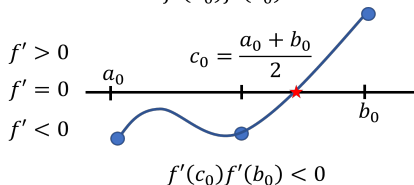
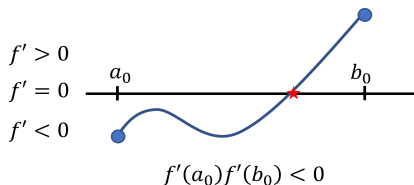
Bisection method: illustration



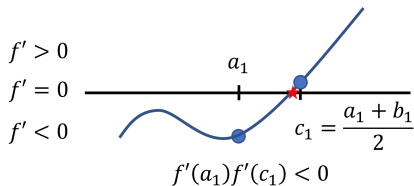
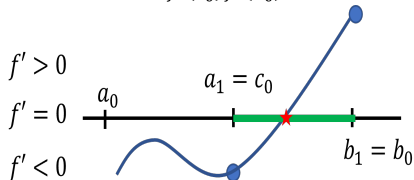
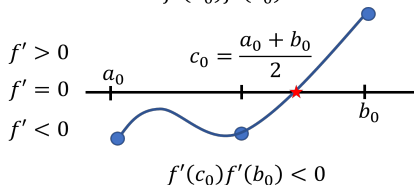
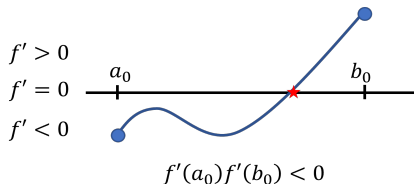
Bisection method: illustration



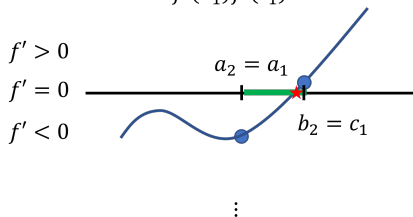
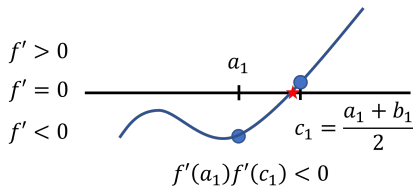
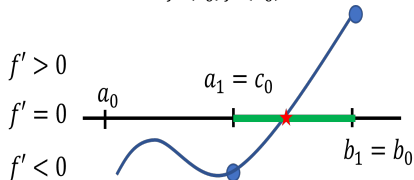
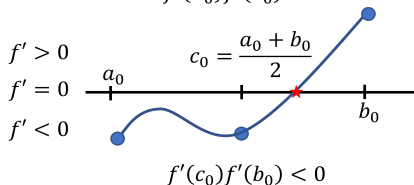
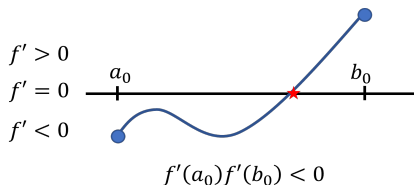
Bisection method: illustration



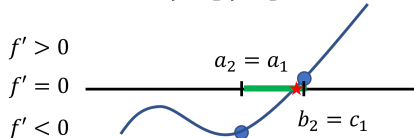
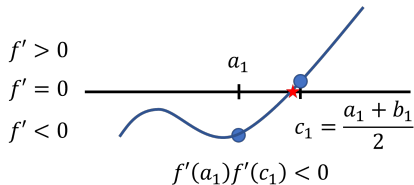
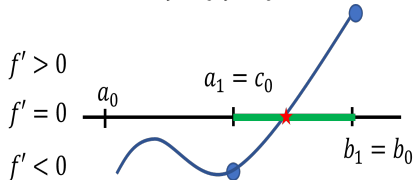
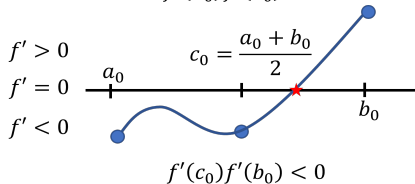
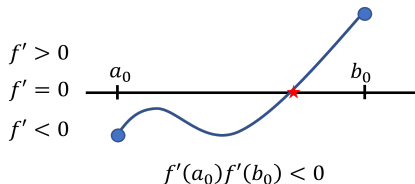
Bisection method: illustration



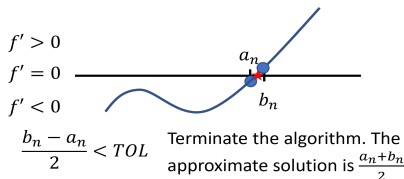
Bisection method: illustration



Bisection method: illustration



⋮



Bisection method: pseudocode

Choose $[a^{(0)}, b^{(0)}]$, $\epsilon_x > 0$, optional $\epsilon_g \geq 0$, and a maximum of $K_{\max}$ reductions.

- 1 Return an endpoint if the absolute value of derivative is smaller than ϵ_g .
- 2 For $k = 0, 1, \dots, K_{\max} - 1$,

- 3 Set

$$x^{(k)} = \frac{a^{(k)} + b^{(k)}}{2}.$$

- 4 If $(b^{(k)} - a^{(k)})/2 \leq \epsilon_x$, return $x^{(k)}$.
- 5 Evaluate $f'(x^{(k)})$. If $|f'(x^{(k)})| \leq \epsilon_g$, return $x^{(k)}$.
- 6 Otherwise retain the half bracket whose endpoint derivatives have opposite signs.

Python code: `bisection_stationary(...)` in `108_one_dim_methods.py`.

Bisection method: common example

$$f(x) = e^x + x^2 - 2x, \quad f'(x) = e^x + 2x - 2, \quad f''(x) = e^x + 2 > 0.$$

- f is strictly convex on $\mathbf{R}$.
- $f'(-1) = e^{-1} - 4 < 0$ and $f'(1) = e > 0$.
- Hence the unique stationary point in $[-1, 1]$ is the unique global minimizer.

We use $\epsilon_x = 10^{-4}$.

Bisection method: common example (cont'd)

k	$a^{(k)}$	$b^{(k)}$	$x^{(k)}$	$f'(x^{(k)})$
0	-1.000000	1.000000	0.000000	-1.000000
1	0.000000	1.000000	0.500000	0.648721
2	0.000000	0.500000	0.250000	-0.215975
3	0.250000	0.500000	0.375000	0.204991

The first three retained brackets are

$$[0, 1], \quad [0, 0.5], \quad [0.25, 0.5].$$

Each sign comparison removes half of the current interval.

Bisection method: common example (cont'd)

k	$a^{(k)}$	$b^{(k)}$	$x^{(k)}$	$f'(x^{(k)})$
0	-1.000000	1.000000	0.000000	-1.000000
1	0.000000	1.000000	0.500000	0.648721
2	0.000000	0.500000	0.250000	-0.215975
3	0.250000	0.500000	0.375000	0.204991

The first three retained brackets are

$$[0, 1], \quad [0, 0.5], \quad [0.25, 0.5].$$

Each sign comparison removes half of the current interval.

The Python run gives

reductions	$\hat{x}$	$ \hat{x} - x^* $	derivative calls
14	0.314880371	4.269×10^{-5}	16

Bisection method: complexity and convergence

- **Per iteration:** $1 \times f'$, sign comparison.
- **Total complexity to ϵ_x :** let $\ell^{(k)} = b^{(k)} - a^{(k)}$. After N reductions,

$$\ell^{(N)} = 2^{-N} \ell^{(0)}, \quad |x^{(N)} - x^*| \leq \frac{\ell^{(N)}}{2}$$

where $x^{(N)}$ is the final midpoint and x^* is the unique point in the nested-bracket intersection. Thus it suffices to take

$$N = \max \left\{ 0, \left\lceil \log_2 \left(\frac{\ell^{(0)}}{2\epsilon_x} \right) \right\rceil \right\}.$$

- Bracket widths: **Q-linear** with factor $1/2$.

Newton method: a local quadratic model

Around $x^{(k)}$, use the **second-order Taylor model**

$$q^{(k)}(x) = f(x^{(k)}) + f'(x^{(k)})(x - x^{(k)}) + \frac{1}{2}f''(x^{(k)})(x - x^{(k)})^2.$$

Its stationarity equation is

$$f'(x^{(k)}) + f''(x^{(k)})(x - x^{(k)}) = 0.$$

If $f''(x^{(k)}) \neq 0$, the solution gives the Newton update

$$x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}.$$

Newton method: a local quadratic model (cont'd)

$$x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}.$$

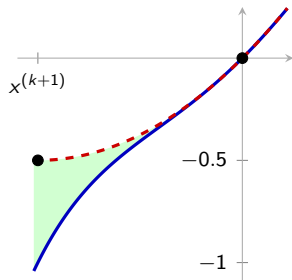
- $f''(x^{(k)}) > 0$: unique global minimizer of $q^{(k)}$.
- $f''(x^{(k)}) < 0$: unique global maximizer of $q^{(k)}$.
- $f''(x^{(k)}) = 0$: the Newton quotient is undefined; if $f'(x^{(k)}) = 0$, stop by the stationarity test first.

A local quadratic model may or may not majorize

At $x^{(k)} = 0$, objectives below have the same second-order Taylor model:

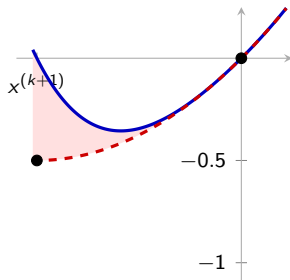
$$q^{(k)}(x) = x + \frac{1}{2}x^2, \quad x^{(k+1)} = -1 \in \arg \min_x q^{(k)}(x).$$

$$f_-(x) = x + \frac{1}{2}x^2 - \frac{1}{2}x^4$$



$$q^{(k)}(x) - f_-(x) = \frac{1}{2}x^4 \geq 0: \text{ majorization}$$

$$f_+(x) = x + \frac{1}{2}x^2 + \frac{1}{2}x^4$$



$$q^{(k)}(x) - f_+(x) = -\frac{1}{2}x^4 \leq 0: \text{ no majorization}$$

Newton method: pseudocode

Choose $x^{(0)} \in \mathcal{D}$, tolerances $\epsilon_g, \epsilon_x > 0$, and $K_{\max}$. Set $k := 0$.

- 1 If $|f'(x^{(k)})| \leq \epsilon_g$, return $x^{(k)}$.
- 2 If $k = K_{\max}$, report the iteration limit. If $|f''(x^{(k)})|$ is too small, stop or safeguard.

- 3 Set

$$d^{(k)} = -\frac{f'(x^{(k)})}{f''(x^{(k)})}, \quad x^{(k+1)} = x^{(k)} + d^{(k)}.$$

- 4 If $|d^{(k)}| \leq \epsilon_x$ return $x^{(k+1)}$; otherwise increase k and go to step 1.

Python code: `newton_stationary(...)` in `108_one_dim_methods.py`.

Newton method: common example

Start at $x^{(0)} = 0$ and stop when $|f'(x^{(k)})| \leq \epsilon_g = 10^{-8}$.

k	$x^{(k)}$	$f(x^{(k)})$	$f'(x^{(k)})$
0	0.000000000	1.000000000	-1.000×10^0
1	0.333333333	0.840056870	6.228×10^{-2}
2	0.314992285	0.839484309	2.333×10^{-4}
3	0.314923059	0.839484301	3.283×10^{-9}

The last estimate has error approximately 9.74×10^{-10} .

Newton method: complexity and convergence

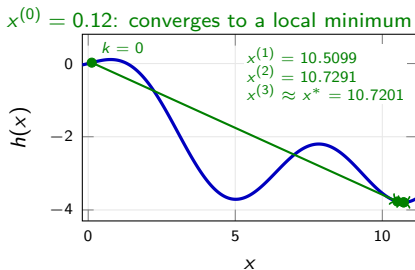
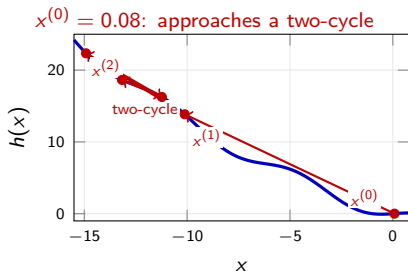
- **Per iteration:** $1 \times f'$, $1 \times f''$, scalar $/$.
- **To remain defined:** $f''(x^{(k)}) \neq 0$ along the generated sequence.
- **For local Q-quadratic convergence:** $f''(x^*) \neq 0$, f'' locally Lipschitz near x^* , and $x^{(0)}$ sufficiently close.
- **For a local minimum:** add $f''(x^*) > 0$.

Pure Newton has no general global convergence or worst-case total complexity guarantee without safeguarding.

Newton method: dependence on initialization

For $h(x) = \sin x - 0.8x + 0.05x^2$, Newton update is

$$x^{(k+1)} = x^{(k)} - \frac{\cos x^{(k)} - 0.8 + 0.1x^{(k)}}{-\sin x^{(k)} + 0.1}.$$



Why the nearby starts split

$0.08 < \arcsin(0.1) \approx 0.1002 < 0.12$, and $h''(\arcsin(0.1)) = 0$.

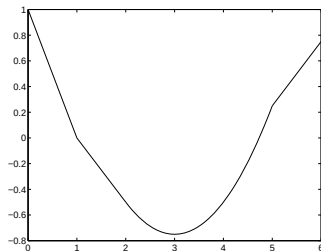
Golden-section search

Golden-section search requires only function values.

Definition 8.2 (Strict unimodality)

A function $f : [a, b] \rightarrow \mathbf{R}$ is strictly unimodal for minimization if it has a unique minimizer x^* , is strictly decreasing on $[a, x^*]$, and is strictly increasing on $[x^*, b]$.

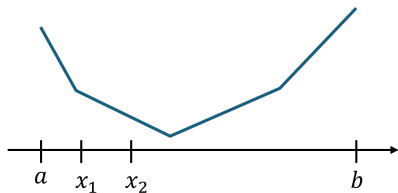
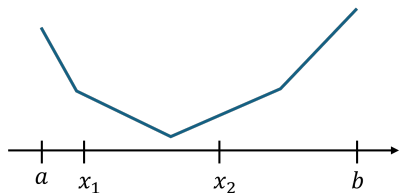
Differentiability is unnecessary.



Golden-section search: illustration

Choose two interior points $a < x_1 < x_2 < b$ and compare $f(x_1)$, $f(x_2)$.

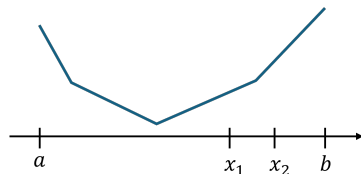
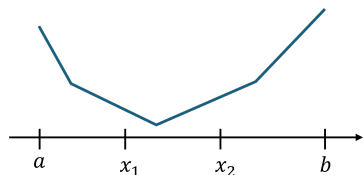
Case (a): $f(x_1) > f(x_2)$ The minimizer cannot lie in $[a, x_1]$. Retain $[x_1, b]$.



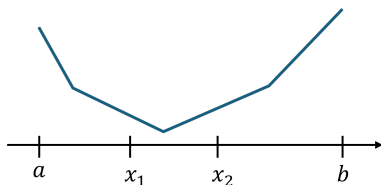
Golden-section search is a derivative-free interval-reduction method: two function values determine which side can be discarded.

Golden-section search: illustration (cont'd)

Case (b): $f(x_1) < f(x_2)$. The minimizer cannot lie in $[x_2, b]$; retain $[a, x_2]$.

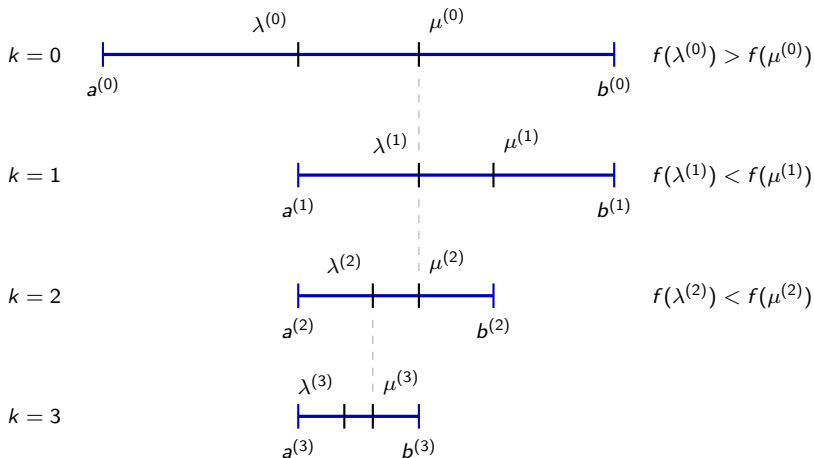


Case (c): $f(x_1) = f(x_2)$. Strict unimodality gives $x^* \in (x_1, x_2)$.



“Golden-section” saves one function value.

Golden-section search: illustration (cont'd)



Blue segments are the retained brackets; each dashed connector marks the interior point reused at the next iteration.

Golden-section search: illustration (cont'd)

Let

$$\varphi = \frac{1 + \sqrt{5}}{2}, \quad \rho = \varphi^{-1} = \frac{\sqrt{5} - 1}{2} \approx 0.618, \quad \rho^2 = 1 - \rho.$$

At iteration k , choose

$$\begin{aligned}\lambda^{(k)} &= b^{(k)} - \rho(b^{(k)} - a^{(k)}), \\ \mu^{(k)} &= a^{(k)} + \rho(b^{(k)} - a^{(k)}).\end{aligned}$$

After discarding one side, one old interior point occupies the correct golden-section position in the new bracket.

Golden-Ratio Day: 18/06; Fibonacci Day: 23/11

Golden-section method: pseudocode

Initialize $[a^{(0)}, b^{(0)}]$, $\epsilon_x > 0$, $K_{\max}$, and the two golden interior points. Set $k := 0$.

- ❶ If $(b^{(0)} - a^{(0)})/2 \leq \epsilon_x$, return the midpoint.
- ❷ For $k = 0, 1, \dots, K_{\max} - 1$,
 - ❸ If $f(\lambda^{(k)}) > f(\mu^{(k)})$, retain $[\lambda^{(k)}, b^{(k)}]$.
 - ❹ Otherwise retain $[a^{(k)}, \mu^{(k)}]$.
 - ❺ Reuse the shared interior value and evaluate f once at the new Golden-section point.
 - ❻ If the new interval size is not larger than $2\epsilon_x$, return the midpoint.

Python code: `golden_section(...)` in `108_one_dim_methods.py`

Golden-section method: common example

For $[-1, 1]$, the initial interior points are

$$\lambda^{(0)} = -0.236068, \quad \mu^{(0)} = 0.236068.$$

Their function values are

$$f(\lambda^{(0)}) = 1.317591, \quad f(\mu^{(0)}) = 0.849853.$$

Hence retain $[-0.236068, 1]$. The next comparison is

point	function value
0.236068	0.849853
0.527864	0.918220

and the next retained bracket is $[-0.236068, 0.527864]$.

Golden-section method: common example

For $[-1, 1]$, the initial interior points are

$$\lambda^{(0)} = -0.236068, \quad \mu^{(0)} = 0.236068.$$

Their function values are

$$f(\lambda^{(0)}) = 1.317591, \quad f(\mu^{(0)}) = 0.849853.$$

Hence retain $[-0.236068, 1]$. The next comparison is

point	function value
0.236068	0.849853
0.527864	0.918220

and the next retained bracket is $[-0.236068, 0.527864]$.

With $\epsilon_x = 10^{-4}$, the Python run gives

reductions	$\hat{x}$	$ \hat{x} - x^* $	function calls
20	0.314935576	1.252×10^{-5}	22

Golden-section method: complexity and convergence

- **Per iteration:** $1 \times f$, scalar comparison.
- **Total complexity to ϵ_x :** let $\ell^{(k)} = b^{(k)} - a^{(k)}$. After N reductions,

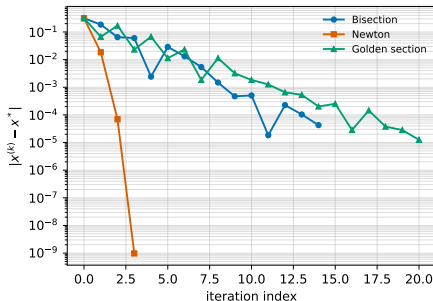
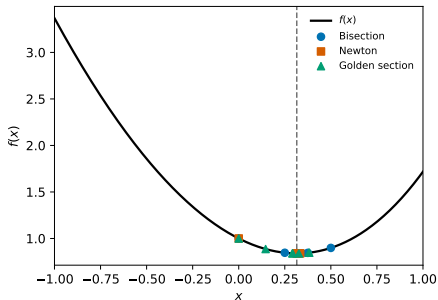
$$\ell^{(N)} = \rho^N \ell^{(0)}, \quad |x^{(N)} - x^*| \leq \frac{\ell^{(N)}}{2}.$$

Thus it suffices to take

$$N = \max \left\{ 0, \left\lceil \frac{\log(\ell^{(0)} / (2\epsilon_x))}{\log(1/\rho)} \right\rceil \right\}.$$

- Bracket widths: **Q-linear** with factor $\rho \approx 0.618$.

The three methods on the same objective



Left: the first few estimates. Right: actual error against the known benchmark solution.
Different algorithms use different stopping information.

Run comparison

Common objective and reference values:

$$x^* = 0.314923057845, \quad f^* = 0.839484300981.$$

Method	updates	$\hat{x}$	$ \hat{x} - x^* $	oracle calls
Bisection	14	0.314880371	4.269×10^{-5}	16 f'
Newton	3	0.314923059	9.742×10^{-10}	4 f' , 3 f''
Golden section	20	0.314935576	1.252×10^{-5}	22 f

This is not a universal ranking: a function, gradient, and Hessian can have very different costs, especially in high dimensions.

Take-home messages

- Initialization + update + termination.
- Line search: one-dimensional subproblem.

Method	Guaranteeing assumptions	Rate	Cost to reach tolerance
Bisection	Continuous f' , sign-changing bracket	Q-linear, factor $1/2$	One new f' per reduction; $O(\log(\ell^{(0)}/\epsilon_x))$
Newton	Regular stationary point, local smoothness, sufficiently close initial point	Locally Q-quadratic	One f' and one f'' per step; no general global bound
Golden section	Strictly unimodal objective on a closed bracket	Q-linear, factor ρ	One new f per reduction after two initial values; $O(\log(\ell^{(0)}/\epsilon_x))$

No method dominates for any setting.